



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования

**«Дальневосточный федеральный университет»
(ДВФУ)**

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математического и компьютерного моделирования

Гречихин Илья Сергеевич
Иванова Ольга Геннадьевна

**КЛАССИФИКАЦИЯ ВИДОВ ДЕРЕВЬЕВ НА ОСНОВЕ МАССИВОВ
ТОЧЕК LiDAR С ПРИМЕНЕНИЕМ НЕЙРОННЫХ СЕТЕЙ**

КУРСОВАЯ РАБОТА

Студенты группы Б9123-02.03.01сцт

Руководитель Амосова Е. В.

Регистрационный № _____

Оценка _____

(подпись)

(И. О. Фамилия)

(подпись)

(И. О. Фамилия)

«__» _____ 2026 г.

«__» _____ 2026 г.

г. Владивосток
2026

Оглавление

Введение.....	3
1 Теоретические основы классификации данных лазерного сканирования LiDAR.....	4
1.1 Применение технологии LiDAR в лесном хозяйстве и экологии	4
1.2 Особенности формата 3D-облаков точек и проблемы их обработки.....	7
1.3 Классическое машинное обучение и конструирование признаков (Feature Engineering).....	9
1.4. Обзор архитектур глубокого обучения для 3D-данных	12
2 Методология и предварительная обработка данных.....	15
2.1 Описание исходного набора данных и программного стека	15
2.2 Фильтрация шумов и алгоритм удаления подстилающей поверхности.....	18
2.3 Прореживание облака точек и нормализация данных	19
2.4. Формирование независимой отложенной выборки для объективного тестирования	21
3. Практическая реализация алгоритмов классификации	22
3.1. Реализация базовой модели: Случайный лес (Random Forest).....	22
3.1.1. Математическое извлечение геометрических характеристик	23
3.1.2. Обучение модели Random Forest.....	25
3.2. Реализация глубокой нейронной сети PointNet++.....	26
3.2.1. Настройка архитектуры и гиперпараметров сети в PyTorch Geometric	26
3.2.2. Процесс градиентного спуска и анализ кривых обучения	28
3.3. Разработка интерактивного программного интерфейса для визуализации результатов.....	29
4. Анализ результатов и сравнение моделей.....	31
4.1. Оценка качества классификации по метрикам Accuracy, Precision, Recall и F1-Score	31
4.2. Визуальный анализ результатов, интерпретация признаков и сравнительная оценка моделей.....	34
Заключение	37

Введение

Цель исследования: адаптация и обучение алгоритмов машинного и глубокого обучения для автоматической классификации видов деревьев Дальнего Востока на основе трехмерных облаков точек лазерного сканирования.

Актуальность исследования: Мониторинг обширных и труднодоступных лесов Дальнего Востока – сложная, но важная задача. Традиционная наземная оценка лесов требует слишком много времени и человеческих ресурсов. Съемка с обычных камер или спутников также имеет ограничения: она показывает только верхнюю часть крон и не всегда позволяет точно определить породу дерева. В связи с этим лесному хозяйству нужны новые методы. Эту проблему успешно решает технология лазерного сканирования *LiDAR*. Она создает детализированные трехмерные копии деревьев в виде облаков точек, которые повторяют уникальную форму стволов и ветвей. Чтобы не анализировать такие огромные массивы данных вручную, в данной работе применяются современные нейронные сети. Они позволяют автоматизировать процесс и с высокой точностью распознавать виды деревьев, что необходимо для грамотного управления лесными ресурсами региона.

Для достижения поставленной цели были сформулированы следующие задачи:

- 1) Подготовить размеченную выборку облаков точек отдельных деревьев для выбранных видов Дальнего Востока.
- 2) Адаптировать и обучить модели классификации, опирающиеся как на классическое машинное обучение (извлечение геометрических признаков), так и на глубокие нейронные сети.
- 3) Сравнить результаты качества категоризации у различных нейросетевых моделей и выделить ту, которая больше подходит для классификации различных видов деревьев.

Объект исследования: Трехмерные массивы точек лазерного сканирования лесных массивов.

Предмет исследования: нейросетевые методы и архитектуры для классификации видов деревьев Дальнего Востока по трёхмерным облакам точек лазерного сканирования.

1 Теоретические основы классификации данных лазерного сканирования LiDAR

1.1 Применение технологии LiDAR в лесном хозяйстве и экологии

Оценка состояния лесных ресурсов исторически считалась одной из самых трудоемких задач природопользования. На протяжении десятилетий учёт лесов опирался преимущественно на ручной труд: специалисты выезжали на пробные площади, измеряли диаметры стволов мерными вилками, определяли высоту высотомерами и визуально оценивали породный состав. Подобный подход не только требовал огромных временных и финансовых затрат, но и неизбежно приводил к статистическим погрешностям при экстраполяции данных на обширные территории.

Появление технологии *LiDAR* значительно упростило и ускорило сбор пространственных данных о лесах. В отличие от обычных камер или спутников, которые просто улавливают отраженный солнечный свет, *LiDAR* работает как активный сенсор. Он сам испускает лазерные лучи и засекает время, за которое луч долетает до объекта и возвращается обратно.

С физической точки зрения принцип работы классического импульсного лидара базируется на измерении времени пролета. Дистанция D от сенсора до точки отражения вычисляется по фундаментальной формуле:

$$D = \frac{c \cdot \Delta t}{2}, \text{ где:}$$

- c – скорость света (приблизительно м/с);
- Δt – время, прошедшее между испусканием импульса и приемом отраженного сигнала;

Деление на 2 обусловлено тем, что луч проходит двойной путь (до объекта и обратно).

Объединение этих данных с системами точного позиционирования (*GPS/GNSS*) и датчиками (*IMU*), которые непрерывно отслеживают любые наклоны и повороты летательного аппарата в воздухе, позволяет превратить простое расстояние от сканера до объекта в точные трехмерные координаты для каждой отраженной точки.

Главное и неоспоримое преимущество лазерного сканирования перед традиционной аэрофотосъемкой или фотограмметрией с БПЛА заключается в способности лазерного луча проникать сквозь просветы в лесной кроне. Один испущенный импульс может дать несколько отражений (возвратов). Первый возврат обычно фиксирует верхушку кроны, промежуточные отражаются от веток и ствола, а последний достигает поверхности земли. Именно эта особенность делает *LiDAR* незаменимым инструментом в лесном хозяйстве.

На сегодняшний день технология решает целый спектр прикладных и научно-исследовательских задач:

1) Построение высокоточных цифровых моделей рельефа. Лазерный луч способен проникать сквозь листву, поэтому лучи, вернувшиеся к сенсору последними, обычно отражаются от самой земли. Это позволяет построить точную 3D-модель рельефа даже под густым лесом. А лучи, вернувшиеся первыми, формируют модель верхушек деревьев. Если вычесть из высоты верхушек высоту земли под ними, мы получим точную высоту самого леса:

$$\text{Высота деревьев} = \text{Высота верхушек} - \text{Высота рельефа}.$$

Благодаря такому простому математическому вычислению алгоритмы могут автоматически находить отдельные деревья и измерять их рост с точностью до сантиметров.

2) Оценка надземной биомассы и запасов углерода. Чтобы рассчитать объем древесины, нужны два главных параметра: высота дерева и диаметр его ствола на уровне груди. Наземные лазерные сканеры могут измерять стволы с точностью до миллиметров. На основе этих данных компьютер представляет сложную форму ствола в виде набора простых цилиндров, что позволяет очень точно высчитать объем древесины. Эта информация необходима экологам, чтобы понимать, какие объемы углекислого газа поглощают и удерживают леса.

3) Мониторинг экологических ниш и биоразнообразия. Трехмерная структура леса – это сложная среда обитания животных. То, насколько плотно сканер фиксирует точки на разных высотах дерева, напрямую отражает густоту его ветвей и листвы. По этим данным можно понять, сколько света и влаги проникает в разные части кроны и насколько эти ярусы подходят для создания гнезд или укрытий птиц и млекопитающих.

4) Классификация породного состава. Это одно из самых сложных и перспективных направлений. Различные виды деревьев обладают уникальной архитектурой ветвления, формой кроны и текстурой коры. Например, хвойные породы чаще имеют выраженную конусовидную форму и плотный центральный ствол вплоть до верхушки. Лиственные породы характеризуются округлой, раскидистой кроной. Лазерное сканирование фиксирует эти геометрические паттерны.

Несмотря на высокую точность самого оборудования, получение данных сканирования – это только первый этап. На выходе лазерный сканер генерирует «сырое» облако, состоящее из десятков и сотен миллионов точек. Изначально эти данные представляют собой просто набор координат без смысловой нагрузки. Компьютер не понимает, к какому объекту относится конкретная точка: где заканчивается поверхность земли и начинается ствол дерева, или где ветки одной ели переплетаются с ветвями соседней осины.

Проблема усугубляется тем, что лесные массивы представляют собой крайне хаотичную и зашумленную среду. Извлечение полезной информации из такого

объема неструктурированных пространственных данных требует применения сложного математического аппарата и современных алгоритмов анализа данных, что и обуславливает переход от простых эвристических методов к инструментам машинного обучения.

1.2 Особенности формата 3D-облаков точек и проблемы их обработки

Результатом работы любой системы лазерного сканирования является цифровой массив данных, получивший название «облако точек». В индустрии пространственных данных фактическим стандартом для хранения и обмена подобной информацией выступает бинарный формат *LAS* (и его сжатая версия *LAZ*).

С математической точки зрения, сырое облако точек представляет собой неупорядоченное дискретное множество S , состоящее из N элементов. Каждый элемент (точка) P_i в этом множестве описывается вектором атрибутов:

$$P_i = (X_i, Y_i, Z_i, I_i, R_i, N_i, C_i), \text{ где}$$

- (X_i, Y_i, Z_i) – декартовы координаты точки в заданном пространстве,
- I_i – интенсивность отраженного сигнала (зависящая от отражательной способности материала, например, коры или листвы),
- R_i – номер текущего возврата,
- N_i – общее количество возвратов для данного импульса,
- C_i – классификационный код.

Несмотря на кажущуюся простоту такого представления, прямая вычислительная обработка облаков точек сопряжена с рядом фундаментальных проблем, которые делают невозможным применение традиционных алгоритмов компьютерного зрения, созданных для двумерных изображений.

Первая и самая критичная проблема – это отсутствие жесткого порядка данных, или так называемая пермутационная инвариантность. Обычная цифровая фотография состоит из пикселей, которые строго привязаны к своей сетке. Если в

коде картинки поменять местами две строки пикселей, изображение визуально сломается. Облако точек, напротив, не имеет внутренней структуры. В памяти компьютера это просто список координат. Если вы случайно перемешаете строчки в этом списке, само трехмерное дерево никак не изменится. Это означает, что нейронная сеть должна уметь правильно распознавать объект независимо от того, в каком порядке точки загружаются в ее память. Классические нейросети, созданные для анализа обычных фотографий, с такой задачей не справляются.

Вторая проблема заключается в неравномерной плотности распределения данных. В 2D-изображениях расстояние между центрами соседних пикселей всегда константно. В данных *LiDAR* плотность точек падает пропорционально квадрату расстояния от сканера до объекта. Верхушка кроны дерева, находящаяся ближе к сенсору БПЛА, будет описана тысячами точек, в то время как нижняя часть ствола, скрытая ветвями, может быть представлена лишь десятком случайных отражений. Это явление усугубляется окклюзиями (перекрытиями) – крупные ветви создают позади себя «слепые зоны», в результате чего сканируемый объект получается неполным, с огромными пространственными дырами.

Третьим фундаментальным ограничением является то, что точки никак не связаны между собой физически. Облако точек показывает, где находится объект в пространстве, но не объясняет, как его части соединены друг с другом. Точки лидара существуют изолированно. Чтобы алгоритм понял форму объекта (например, осознал, что конкретная группа точек – это цилиндрическая ветка), ему нужно постоянно вычислять расстояния между соседними точками по классической формуле расстояния:

$$d(P_i, P_j) = \sqrt{(X_i - X_j)^2 + (Y_i - Y_j)^2 + (Z_i - Z_j)^2}$$

Проблема в том, что компьютер начнет вычислять расстояние по принципу «каждая точка с каждой» для облака из миллионов элементов, а это потребует астрономического количества вычислений. В результате оперативная память переполняется, а обработка занимает слишком много времени. Чтобы избежать

этого, перед анализом данные специальным образом группируют и строят пространственные деревья поиска, которые подсказывают компьютеру, где искать ближайшие точки, экономя вычислительные ресурсы.

Наконец, нельзя игнорировать огромный объем данных и наличие шумов. Один пролет дрона с *LiDAR*-системой способен сгенерировать гигабайты данных за несколько минут. Внутри этого массива неизбежно присутствуют выбросы – ложные отражения от птиц, атмосферных осадков, пыли или так называемый многолучевой эффект.

Совокупность этих факторов (неупорядоченность, пустоты, неравномерность и колоссальный масштаб) исторически заставляла исследователей прибегать к компромиссным решениям. Чаще всего облака точек проецировали на 2D-плоскость, создавая растровые карты высот, либо принудительно разбивали 3D-пространство на кубические элементы. Оба подхода вели к необратимой потере исходной детализации. Разработка методов, способных потреблять сырые, неструктурированные координаты напрямую, стала главным вызовом для специалистов в области анализа пространственных данных в последние несколько лет.

1.3 Классическое машинное обучение и конструирование признаков (Feature Engineering)

До активного внедрения нейронных сетей для классификации деревьев по данным *LiDAR* применялось классическое машинное обучение. Такие алгоритмы, как Метод опорных векторов (*SVM*), Метод К-ближайших соседей и алгоритмы на основе решающих деревьев (*Gradient Boosting*, *Random Forest*), показывали хорошие и стабильные результаты.

Однако фундаментальной проблемой оставался тот факт, что ни один из этих алгоритмов не способен воспринимать сырое облако точек как осмысленный объект. Классические классификаторы ожидают на вход жестко структурированную двумерную таблицу, где каждая строка представляет один

объект наблюдения, а каждый столбец несет конкретную физическую или статистическую интерпретацию. Следовательно, между сырыми 3D-координатами и алгоритмом классификации должен располагаться промежуточный математический слой преобразования.

Этот этап называется конструированием признаков (*Feature Engineering*). Его суть заключается в том, что исследователь с помощью формул вручную извлекает полезные характеристики из облака точек. Успех таких классических моделей классификации во многом зависит от того, насколько удачные параметры смог придумать и запрограммировать человек.

Исторически сформировалось несколько семейств признаков, описывающих морфологию дерева:

1) Макрогеометрические признаки (Базовые габариты).

Самые очевидные характеристики, описывающие общие границы объекта. К ним относятся абсолютная высота дерева H (разница между максимальной и минимальной координатой Z), ширина кроны W_x и W_y (размах точек по осям X и Y), а также различные производные коэффициенты, например, индекс удлиненности кроны (H/W_{mean}).

2) Статистические признаки пространственного распределения.

Эти метрики описывают, как именно масса точек распределена внутри габаритного объема дерева. Они позволяют дифференцировать деревья с плотной, монолитной кроной от деревьев с рыхлым, ажурным ветвлением.

Важнейшим инструментом здесь выступают перцентили распределения высот. Перцентиль P_k показывает высоту, ниже которой располагается $k\%$ всех точек кроны. Расчет набора перцентилей (например, $P_{10}, P_{25}, P_{50}, P_{75}, P_{90}$) фактически строит одномерный профиль дерева. Например, у ели, имеющей широкое основание кроны и сужающуюся верхушку, медианная высота (P_{50}) будет смещена ближе к земле. У сосны обыкновенной, теряющей нижние ветви с

возрастом, масса точек сконцентрирована в верхней трети, что сдвинет значения перцентилей вверх.

Помимо перцентилей, активно применяются моменты распределения координат:

- Дисперсия по трем осям, характеризующая общую степень разброса:
- $\sigma_Z^2 = \frac{1}{N} \sum_{i=1}^N (Z_i - \bar{Z})^2$
- Асимметрия, показывающая перекося плотности в верхнюю или нижнюю часть кроны.
- Эксцесс, отражающий наличие экстремально густых скоплений точек.

3) Анализ локальной размерности и тензор ковариации (РСА)

Чтобы алгоритм понял, на что похожа небольшая группа соседних точек (на вытянутую ветку, плоский лист или объемный куст), используется математический анализ их формы. Для этого вычисляется матрица ковариации $C \in \mathbb{R}^{3 \times 3}$:

$$C = \frac{1}{N} \sum_{i=1}^N (P_i - \bar{P})(P_i - \bar{P})^T$$

где:

- P_i – координаты точки,
- $\bar{P}$ – центр масс рассматриваемого облака.

Решая это математическое уравнение, мы получаем три основных числа (собственные значения $\lambda_1 \geq \lambda_2 \geq \lambda_3$). По соотношению этих чисел можно легко описать физическую форму объекта:

- Если $\lambda_1 \approx \lambda_2 \gg \lambda_3$, точки лежат в плоскости (2D структура, характерная для поверхности земли или крупных листьев).
- Если $\lambda_1 \approx \lambda_2 \approx \lambda_3$, точки распределены в объеме (3D структура, характерная для пересечения ветвей внутри густой кроны).

Комбинируя эти значения, можно получить инвариантные признаки, такие как линейность, планарность и сферичность облака.

Сформированный вектор признаков подается на вход ансамблевым методам машинного обучения. Алгоритм Случайного леса (*Random Forest*) исторически показал наилучшие результаты в этой задаче. Его суть заключается в построении множества независимых решающих деревьев. Каждое такое дерево в процессе обучения пытается найти оптимальные пороги разделения в пространстве признаков. Например, узел дерева может выработать правило: *«Если высота больше 15 метров, а P_{50} меньше 7 метров, то это дерево хвойной породы»*.

Финальный ответ алгоритма формируется путем простого голосования всех построенных деревьев. Ключевым достоинством *Random Forest* является его высокая устойчивость к переобучению, умение работать с коррелирующими признаками и возможность оценки важности каждого придуманного нами признака, что делает эту модель хорошо интерпретируемой.

Однако у ручного конструирования признаков есть существенный минус. Формулы, заданные человеком, описывают дерево лишь в общих чертах. Они хорошо передают общую статистику кроны, но упускают из виду микроструктуру – уникальный рисунок ветвей или текстуру коры, которые нельзя выразить простыми уравнениями. Из-за этого возникла потребность в методах, которые могут самостоятельно находить важные закономерности прямо в координатах исходных точек, что и стало причиной перехода к нейронным сетям глубокого обучения.

1.4. Обзор архитектур глубокого обучения для 3D-данных

Успешное применение сверточных нейронных сетей (*CNN*) для распознавания обычных фотографий подтолкнуло ученых использовать их и для 3D-данных. Однако прямое применение классических сверток к облакам точек невозможно из-за уже упомянутой проблемы пермутационной инвариантности и отсутствия регулярной сетки.

Эволюция подходов к применению глубокого обучения для работы с *LiDAR*-данными прошла через три концептуально разных этапа: вокселизация, проекционные методы и, наконец, прямое потребление точек.

Воксельные архитектуры стали первой попыткой решить проблему нерегулярности. Облако точек принудительно помещалось в регулярную трехмерную сетку. Пространство разбивалось на элементарные кубики (воксели). Если внутрь вокселя попадала хотя бы одна точка *LiDAR*, ему присваивалось значение «1» (занято), иначе – «0» (пусто). К полученной матрице применялись 3D-сверточные нейросети.

Главным минусом этого метода стали огромные затраты вычислительных ресурсов и памяти. Крона дерева – это в основном пустое пространство между ветвями. Если разбить ее на воксели, более 90% из них окажутся пустыми. В результате компьютеру приходилось тратить мощности впустую, обрабатывая пустоту.

Альтернативой стали мультивидовые проекции (*Multi-view CNN*). Суть подхода заключалась в «программном фотографировании» 3D-объекта с разных ракурсов. Полученные 2D-изображения (обычные проекции или карты глубины) передавались в нейросети, созданные для анализа фотографий (например, *ResNet* или *VGG*). Затем данные со всех ракурсов объединялись. Этот метод отлично работает для распознавания искусственных объектов (мебели, автомобилей), но для лесного хозяйства он не подошел. Дерево имеет слишком сложную структуру: при попытке сделать плоскую картинку ветви перекрывают друг друга, и значительная часть информации о форме просто теряется.

Важным шагом в анализе 3D-данных стало создание архитектуры *PointNet* в 2017 году. Это была первая нейросеть, способная обрабатывать неструктурированные точки напрямую, без их превращения в кубики или плоские картинки. Чтобы решить проблему отсутствия порядка у точек, в *PointNet*

применили операцию выбора максимального значения (*max-pooling*). Сеть работает следующим образом:

1) Каждая точка пространства (X, Y, Z) обрабатывается индивидуально через небольшую базовую нейросеть (*MLP*). Это позволяет преобразовать простые координаты каждой точки в сложный набор из 1024 признаков, описывающих ее геометрические свойства.

2) После того как все N точек превращены в N векторов признаков, применяется операция глобального максимизирующего пулинга (*Global Max Pooling*). Из каждого из 1024 столбцов выбирается максимальное значение.

3) На выходе формируется единый вектор (размером 1024), который является глобальным «цифровым отпечатком» всего объекта. Порядок точек во входном массиве больше не имеет значения, так как операция нахождения максимума коммутативна.

При всех своих достоинствах базовая модель *PointNet* имела ограничение: она видела либо каждую точку по отдельности, либо форму всего дерева целиком. Сеть плохо понимала локальные детали и не могла качественно распознать, как именно группа соседних точек формирует отдельную ветку.

Для решения этой проблемы та же группа исследователей представила *PointNet++* – архитектуру, реализующую принцип иерархического извлечения признаков. Сеть работает подобно сверткам на изображениях, постепенно расширяя рецептивное поле.

Основой *PointNet++* является поэтапная группировка и анализ точек. Этот процесс состоит из трех шагов:

1) *Sampling* (Выборка): программа равномерно раскидывает по облаку точек «якоря» – выбирает отдельные точки, которые станут центрами локальных областей.

2) *Grouping* (Группировка): вокруг каждого центрального узла очерчивается сфера заданного радиуса. Все точки, попавшие внутрь сферы, объединяются в локальный кластер (соседство).

3) *PointNet*: каждый такой кластер локально обрабатывается мини-версией оригинальной сети *PointNet*, сжимая информацию о соседях в один емкий вектор признаков для центральной точки.

Этот процесс повторяется несколько раз, с каждым слоем уменьшая количество точек, но увеличивая радиус охвата. Таким образом, *PointNet++* способен улавливать как тонкие детали (текстуру коры или мелкие ветви на первых слоях), так и макроструктуру (раскидистость кроны на глубоких слоях).

Параллельным и весьма перспективным направлением развития стали графовые сверточные нейросети (*Graph Convolutional Networks – GCN*), адаптированные для 3D.

В таких сетях облако точек представляется как математическая сеть (граф), где каждая точка соединена невидимыми линиями со своими ближайшими соседями. Вместо того чтобы смотреть на абсолютные координаты каждой точки, графовая сеть изучает отношения между ними: насколько соседние точки отличаются друг от друга. Это позволяет алгоритму легко понимать сложную форму поверхности, выявлять изгибы ствола или углы излома ветвей. Графовые сети показывают отличные результаты, однако у них есть слабое место: алгоритму приходится постоянно заново искать соседей для каждой точки, что требует огромных затрат вычислительной мощности, порой превосходящих аппетиты архитектуры *PointNet++*.

2 Методология и предварительная обработка данных

2.1 Описание исходного набора данных и программного стека

Качество и объем исходных данных – ключевые факторы для успешной работы алгоритмов машинного обучения. Для задачи классификации пород мы собрали базу облаков точек лазерного сканирования, на которых запечатлены

типичные виды деревьев из смешанных лесов умеренного климата. Исходные данные хранятся в бинарном формате «.las».

Датасет логически разделен на несколько поддиректорий, каждая из которых соответствует определенному биологическому виду древесной растительности. В рамках данной работы рассматриваются следующие классы:

- Хвойные породы: Сосна (*Pine*), Ель (*Spruce*), Пихта (*Fir*). У этих деревьев центральный ствол четко выражен от корней до самой верхушки (моноподиальное ветвление). Благодаря этому и высокой плотности хвои крона обычно принимает форму конуса или цилиндра.
- Лиственные породы: Береза (*Birch*), Осина (*Aspen*), Ольха (*Alder*). У этих деревьев центральный ствол на определенной высоте часто расходится на крупные ветви (симподиальное ветвление). В результате крона становится раскидистой, приобретая форму купола или шара.

Каждый отдельный файл «.las» в директории представляет собой сегментированное (вырезанное из общего лесного массива) облако точек, соответствующее одному единственному дереву. Важно отметить, что сырые данные обладают следующими одинаковыми свойствами:

1) Вариативность плотности: Количество точек в файлах варьируется от двух тысяч до нескольких тысяч. Это обусловлено разным расстоянием от сканера до объекта в момент съемки, а также физическими размерами самого дерева.

2) Наличие подстилающей поверхности: ввиду особенностей алгоритмов автоматической сегментации лесного полога, подавляющее большинство файлов содержит не только биомассу самого дерева, но и фрагмент рельефа, на котором оно произрастает, а также потенциальный шумовой подлесок.

3) Отсутствие регулярности: Координаты (X , Y , Z) заданы в произвольной локальной или глобальной системе координат. Точки начала отсчета для разных файлов не совпадают, что делает невозможным их прямое математическое сравнение до этапа пространственной нормализации.

Разработка алгоритмов, предобработка данных, обучение моделей и визуализация результатов выполнялись на языке программирования *Python* (версии 3.10).

Для решения специфических задач в рамках проекта применялись следующие программные библиотеки и модули:

- *laspy*: специализированная библиотека для эффективного парсинга, чтения и записи заголовочной информации и бинарных данных формата «*las*». Используется для извлечения сырых матриц координат.

- *NumPy*: базовая библиотека для научных вычислений. В проекте она применяется для быстрой обработки многомерных массивов и математических расчетов (например, вычисления перцентилей, дисперсии и центров масс).

- *Scikit-learn (sklearn)*: библиотека для классического машинного обучения. Использовалась для создания модели «Случайного леса» (*Random Forest*), а также для подсчета метрик точности (*Accuracy, Precision, Recall*) и построения матриц ошибок (*Confusion Matrix*).

- *PyTorch*: фреймворк для построения нейронных сетей глубокого обучения. Позволяет эффективно работать с тензорами и автоматически переносит вычисления на видеокарты с использованием технологии NVIDIA CUDA.

- *PyTorch Geometric (PyG)*: специализированное расширение для *PyTorch*, созданное для обработки сложных пространственных структур (графов и 3D-облаков точек). Библиотека содержит готовые быстрые алгоритмы для поиска соседних точек и группировки данных, которые необходимы для правильной работы архитектуры *PointNet++*.

- *Matplotlib, Seaborn* и *ipywidgets*: комплекс библиотек для научной визуализации и создания интерактивных графических пользовательских интерфейсов непосредственно в среде *Jupyter Notebook*. Применяются для отрисовки трехмерных проекций облаков точек, тепловых карт и выпадающих меню.

Вычисления и обучение нейросетевых моделей производились с аппаратным ускорением на базе графического процессора *GPU NVIDIA* с поддержкой программной платформы *CUDA 11.8*.

2.2 Фильтрация шумов и алгоритм удаления подстилающей поверхности

Любые «сырые» данные сканирования неизбежно содержат лишнюю информацию. При классификации деревьев главным «шумом» выступает земля, камни, низкий кустарник и трава, которые попали в кадр вместе с нужным деревом.

Наличие земли вокруг ствола искажает данные о форме дерева. Например, если у одной березы сканер захватил плоский участок земли, а у другой – часть холма, алгоритм может ошибочно решить, что перед ним деревья с разной структурой. Кроме того, земля смещает математический центр масс объекта вниз, что мешает дальнейшей нормализации данных. Поэтому удаление точек поверхности – обязательный первый шаг подготовки.

В профессиональных программах для этого используют сложные алгоритмы (например, *Cloth Simulation Filter*), но они требуют слишком много вычислительных ресурсов и избыточны для одиночных деревьев, где рельеф представлен лишь небольшим фрагментом у основания ствола.

В рамках данной работы был применен простой, но надежный алгоритм фильтрации земли (*Threshold-based Ground Filtering*). Его суть заключается в анализе распределения точек по вертикальной оси (высоте Z).

Процесс отсекаания земли реализуется в три математических шага:

1) Чтение и формирование матрицы. Сырой файл «*.las*» считывается с помощью библиотеки *laspy*. Извлекаются массивы координат по трем осям, которые объединяются (через операцию вертикального стека *np.vstack* и транспонирование) в единую плоскую матрицу $P \in \mathbb{R}^{N \times 3}$, где N – общее количество точек в файле.

2) Поиск глобального минимума. В сформированной матрице отыскивается точка с минимальным значением координаты Z . Эта координата принимается за условный «ноль» рельефа для данного конкретного дерева:

$$Z_{min} = \min (P_{:,2})$$

3) Применение пороговой маски. Задается эмпирически подобранный параметр порога T_{ground} (в нашем случае $GROUND_THRESHOLD = 0.4$ метра). Формируется булева маска M , которая принимает значение ИСТИНА только для тех точек, высота которых строго больше суммы минимальной отметки и заданного порога:

$$M_i = P_{i,2} > (Z_{min} + T_{ground})$$

К исходному облаку точек применяется фильтрация по маске. В результате мы получаем облако точек, которое содержит исключительно ствол и крону дерева без элементов рельефа.

2.3 Прореживание облака точек и нормализация данных

После успешного удаления подстилающей поверхности мы получаем очищенное облако точек, представляющее геометрию дерева. Однако эти данные все еще обладают двумя фундаментальными недостатками, препятствующими их использованию в алгоритмах машинного и глубокого обучения: во-первых, деревья имеют разное количество точек; во-вторых, их координаты заданы в разных локальных системах и абсолютных размерах. Решение этих проблем достигается через процедуры прореживания и нормализации.

Нейронные сети (включая *PointNet* и *PointNet++*) принимают данные блоками (батчами) строго одинакового размера. Компьютер не может одновременно обрабатывать одно дерево из 3500 точек, а другое из 12000. Все объекты нужно привести к единому стандарту, где N – это фиксированное количество точек. В нашем проекте это значение было установлено на $N=2048$ точек. Это оптимальный компромисс: такого количества достаточно, чтобы сохранить рисунок ветвей, но при этом оно не перегружает память видеокарты во время обучения.

Приведение размера облака к константе осуществляется алгоритмически в зависимости от исходного количества точек N_{orig} :

- Уменьшение количества точек. Если в облаке больше 2048 точек, лишние нужно удалить. Для этого программа просто выбирает 2048 случайных точек, а остальные отбрасывает. Поскольку лазерный сканер изначально собирает данные с большим запасом, удаление части точек делает облако менее плотным, но общая форма дерева при этом полностью сохраняется.
- Увеличение количества точек. Если из-за помех или низкой плотности сканирования точек оказалось меньше 2048, программе нужно добавить недостающие. Для этого она берет уже существующие точки и создает их точные копии, пока общее число не достигнет 2048. Для 3D-нейросетей (таких как *PointNet*) наличие дубликатов не создает проблем: математика сети устроена так, что при анализе она просто игнорирует повторяющиеся координаты.

Исходные координаты точек часто имеют вид реальных географических координат или просто смещены от нуля. Если напрямую загрузить такие огромные числа в нейросеть, процесс обучения сломается из-за математических ограничений алгоритма. К тому же, для классификации важна именно форма дерева и его пропорции, а не реальные размеры в метрах или точное положение в лесу.

Нормализация каждого отдельного облака точек выполняется в два этапа, сначала происходит центрирование: вычисляется центр масс облака μ путем нахождения среднего арифметического по каждой из осей:

$$\mu = \left(\frac{1}{N} \sum_{i=1}^N X_i, \frac{1}{N} \sum_{i=1}^N Y_i, \frac{1}{N} \sum_{i=1}^N Z_i \right)$$

Затем из координат каждой точки вычитается вектор центроида:

$$P_i^{centered} = P_i - \mu$$

После этой операции дерево «переносится» ровно в точку (0, 0, 0).

Далее происходит процесс масштабирования в единичную сферу. Чтобы нейросеть воспринимала все объекты в едином математическом масштабе, облако

сжимается так, чтобы его самая дальняя точка от центра не выходила за пределы сферы радиусом $R = 1$. Сначала вычисляется максимальное евклидово расстояние D_{max} от нового (нулевого) центра до самой удаленной точки:

$$D_{max} = \max_{1 \leq i \leq N} \left(\sqrt{X_i^2 + Y_i^2 + Z_i^2} \right)$$

Затем все координаты делятся на этот скаляр:

$$P_i^{normalized} = \frac{P_i^{centered}}{D_{max}}$$

В результате этих преобразований каждое дерево переносится в математический «ноль» координат и сжимается так, чтобы поместиться в виртуальную сферу радиусом 1. При этом пропорции дерева (соотношение ширины и высоты) остаются неизменными. Именно эти подготовленные стандартизированные массивы размером 2048 точек сохраняются в файлы формата «.пру» и передаются нейросетям.

2.4. Формирование независимой отложенной выборки для объективного тестирования

Одной из главных проблем машинного обучения является переобучение. Нейросеть может просто «запомнить» особенности конкретных деревьев из тренировочного набора и показать точность около 100%, но при этом не справиться с распознаванием того же вида дерева на новых данных.

Стандартным подходом к решению этой проблемы является разделение исходного набора данных на обучающую (*Train*) и валидационную (*Validation*) выборки. Обучение происходит на первой части, а оценка качества – на второй. Однако в задачах, связанных со сложными пространственными данными (такими как 3D-сканирование леса), такого разделения часто бывает недостаточно.

Причина кроется в феномене «утечки данных» при подборе гиперпараметров. Исследователь, стремясь улучшить метрики на валидационной выборке, многократно меняет настройки модели (в случае *Random Forest* –

количество деревьев или глубину, в случае *PointNet++* – радиус поиска соседей, коэффициент сэмплирования, скорость обучения). В результате этих итераций модель косвенно «подстраивается» под валидационную выборку. Финальная точность, полученная на этих данных, неизбежно оказывается завышенной и не отражает истинную обобщающую способность алгоритма на абсолютно новых данных.

Чтобы симитировать работу алгоритма в реальных условиях леса и получить честные результаты, в рамках проекта была выделена полностью независимая тестовая (отложенная) выборка. Эти данные хранятся изолированно: они никогда не показываются нейросети в процессе обучения и не используются исследователем для подбора настроек. Итоговые проверки точности проводились исключительно на этих скрытых файлах. Такой подход гарантирует, что нейросеть действительно научилась понимать пространственную структуру разных пород деревьев, а не просто заучила конкретные примеры из тренировочного набора.

3. Практическая реализация алгоритмов классификации

3.1. Реализация базовой модели: Случайный лес (Random Forest)

Перед тем как применять сложные глубокие нейросети, необходимо зафиксировать базовый уровень точности. Для этой роли отлично подходит алгоритм случайного леса (*Random Forest*). Он не требует идеальной подготовки данных, редко переобучается и хорошо находит скрытые закономерности.

Главная проблема заключается в том, что классические алгоритмы не понимают трехмерное пространство. Они не могут обработать просто набор точек. Поэтому работу пришлось разделить на два этапа: сначала мы математически описываем форму дерева понятными для компьютера числами, а уже затем передаем эти числа в алгоритм для обучения.

3.1.1. Математическое извлечение геометрических характеристик

Для каждого облака точек, состоящего из $N = 2048$ нормализованных координат (X, Y, Z) , программный алгоритм вычисляет одномерный вектор, состоящий из 17 количественных признаков, исчерпывающе описывающих макро- и микрогеометрию дерева. Выбор метрик базировался на дендрологических особенностях строения различных пород.

Весь набор извлеченных признаков разделен на пять логических групп:

1) Макрогабариты (3 признака). Описывают абсолютные границы кроны по трем декартовым осям. Рассчитываются как разность между максимальным и минимальным значениями соответствующих координат в массиве:

$$H = Z_{max} - Z_{min}$$

$$W_x = X_{max} - X_{min}$$

$$W_y = Y_{max} - Y_{min}$$

2) Дисперсия распределения точек (3 признака). Статистическая мера разброса точек относительно центра масс кроны. Различные породы имеют разную плотность листвы и ветвей. Вычисляется по формуле выборочной дисперсии для каждой оси:

$$Var(X) = \frac{1}{N} \sum_{i=1}^N (X_i - \bar{X})^2$$

Аналогичные вычисления производятся для осей Y и Z .

3) Перцентили распределения по высоте (5 признаков). Эти метрики описывают распределение точек по высоте кроны. Перцентиль P_k возвращает высоту, ниже которой находится ровно $k\%$ всех точек облака. Алгоритм высчитывает пять перцентилей: $P_{10}, P_{25}, P_{50}, P_{75}, P_{90}$. Эти значения помогают модели понимать разницу между породами: например, у ели основная масса веток находится внизу,

поэтому значение P_{50} будет низким, а у сосны крона сосредоточена наверху, из-за чего значение P_{50} будет значительно выше.

4) Плотность распределения по ярусам (3 признака). Общая высота дерева H делится на три равных горизонтальных яруса. Сначала вычисляются границы ярусов:

$$h_{33} = Z_{min} + H \cdot 0.33$$

$$h_{66} = Z_{min} + H \cdot 0.66$$

Затем алгоритм подсчитывает долю точек, попавших в нижний, средний и верхний ярус соответственно. Например, для нижнего яруса:

$$Ratio_{bottom} = \frac{1}{N} \sum_{i=1}^N [Z_i < h_{33}]$$

(где квадратные скобки обозначают индикаторную функцию, равную 1 при выполнении условия и 0 в противном случае).

5) Главные компоненты и собственные значения (3 признака). Этот метод используется для математического описания формы облака точек. Вычисляется матрица ковариации $C \in \mathbb{R}^{3 \times 3}$ для всех точек дерева:

$$C = \frac{1}{N} \sum_{i=1}^N (P_i - \bar{P})(P_i - \bar{P})^T$$

Из этой матрицы извлекаются собственные значения $\lambda_1, \lambda_2, \lambda_3$. Алгоритм сортирует их по убыванию ($\lambda_1 \geq \lambda_2 \geq \lambda_3$). Соотношение этих чисел позволяет математически описать форму дерева: например, понять, похожа ли крона на шар, плоскую тарелку или вытянутый овал.

В результате работы программы исходное облако из 2048 точек заменяется строкой из 17 чисел, описывающих математическую форму дерева. Эта операция поочередно применяется ко всем файлам, формируя готовую обучающую таблицу признаков $X_{features}$

3.1.2. Обучение модели Random Forest

Подготовленная матрица признаков $X_{features}$ и соответствующий ей вектор истинных меток классов y передаются в модуль разделения выборки библиотеки *scikit-learn*. Данные разделяются на обучающую (*Train*) и внутреннюю тестовую (*Test*) выборки в классической пропорции 80% на 20%.

Ключевым параметром функции разделения *train_test_split* в нашем коде выступает аргумент *stratify=y*. Стратификация гарантирует, что исходное распределение классов деревьев (процентное соотношение берез, осин, елей и т.д.) будет в точности сохранено как в обучающем, так и в тестовом наборе. Это предотвращает возникновение ситуаций, когда редкий класс деревьев случайно попадает исключительно в тестовую выборку, делая его распознавание невозможным.

Модель обучалась с помощью класса *RandomForestClassifier*. Количество деревьев в ансамбле (параметр *n_estimators*) было установлено на 200. Это оптимальное значение, которое обеспечивает высокую точность прогнозов и при этом не сильно замедляет процесс обучения. Чтобы результаты эксперимента можно было воспроизвести при повторном запуске кода, использовался параметр *random_state*. Для максимального ускорения работы был включен параметр *n_jobs = -1*, который позволяет задействовать все ядра процессора компьютера.

Обучение модели заключается в том, что алгоритм строит 200 независимых деревьев, каждое из которых обучается на случайной подвыборке данных и использует случайное подмножество из 17 наших признаков для поиска оптимальных порогов разделения. По завершении обучения классификатор сохраняется в оперативной памяти для последующей интеграции с интерактивными виджетами предсказания и финального тестирования на отложенной выборке.

3.2. Реализация глубокой нейронной сети PointNet++

Классические алгоритмы требуют, чтобы человек сам придумывал математические формулы для описания формы дерева. Нейронные сети глубокого обучения работают иначе: они способны самостоятельно находить важные закономерности прямо в исходных координатах точек. Для автоматического извлечения таких признаков в данной работе была выбрана архитектура *PointNet++*.

Сама нейросеть создавалась с помощью фреймворка *PyTorch* и его специального дополнения *PyTorch Geometric*. Эта библиотека была выбрана потому, что в ней уже есть оптимизированные инструменты для сложных пространственных расчетов, например, для быстрого поиска соседних точек. Без этих алгоритмов обучение сети шло бы слишком медленно.

3.2.1. Настройка архитектуры и гиперпараметров сети в PyTorch Geometric

Архитектура *PointNet++* основана на постепенном увеличении области обзора. Этот принцип схож с работой обычных нейросетей для анализа фотографий, но вместо квадратных фильтров здесь применяются так называемые модули абстракции множеств.

Программно сеть состоит из двух последовательных модулей абстракции и финального блока классификации. В ходе экспериментов архитектура сети была усложнена и расширена по сравнению со стандартными решениями. Это было сделано для того, чтобы увеличить «объем памяти» модели и позволить ей запоминать больше сложных геометрических узоров.

Каждый модуль абстракции выполняет три математические операции:

- 1) Выборка. Из всего облака выбирается набор опорных точек. Программа старается выбирать точки так, чтобы они находились как можно дальше друг от друга, обеспечивая равномерное покрытие всего дерева.

2) Группировка. Вокруг каждой опорной точки очерчивается сфера заданного радиуса, и все попавшие в нее точки объединяются в локальную группу.

3) Локальная нейросеть. К каждой такой группе применяется небольшая внутренняя нейросеть. Она изучает взаимное расположение соседних точек и сжимает информацию о них в единый набор признаков для центральной опорной точки.

В нашей работе сеть имеет следующую структуру:

- Первый уровень абстракции: на вход поступает нормализованное облако из 2048 точек. Алгоритм оставляет ровно половину из них – 1024 опорные точки. Для каждой ищутся соседи в небольшом радиусе (0.2). Это позволяет сети сфокусироваться на микроструктуре: изучить изгибы ветвей или плотность хвои. Внутренняя нейросеть преобразует исходные координаты в набор из 256 признаков через каскад слоев: $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$. После каждого слоя данные нормализуются и проходят через функцию активации.

- Второй уровень абстракции: принимает 1024 точки с их признаками и сокращает количество опорных центров до 256. Радиус обзора при этом увеличивается вдвое (до 0.4). На этом этапе сеть анализирует уже целые фрагменты кроны. Пространство признаков расширяется еще сильнее: от 259 к 512.

- Глобальное объединение: специальная функция собирает данные от оставшихся 256 фрагментов, создавая один итоговый вектор из 512 чисел. Этот вектор и является цифровым слепком всей формы дерева.

- Классификатор: итоговый вектор проходит через обычные полносвязные слои, постепенно сужаясь от 512 к 256, а затем к количеству наших пород деревьев. Чтобы модель не зазубривала данные, используется метод случайного отключения нейронов (*Dropout*). В процессе обучения он временно отключает половину связей, заставляя алгоритм каждый раз находить новые закономерности в форме дерева.

3.2.2. Процесс градиентного спуска и анализ кривых обучения

Для того чтобы необученная сеть научилась отличать березу от ели, был запущен итеративный процесс машинного обучения. Данные подавались в сеть не поштучно, а пакетами по 32 дерева одновременно. Это ускоряет расчеты на видеокарте и делает процесс обучения более плавным.

В качестве показателя ошибки, который сеть старается свести к минимуму, использовалась функция перекрестной энтропии. Она оценивает, насколько предсказание сети отличается от правильного ответа.

Важной частью нашей работы стала борьба с переобучением. Если просто показывать сети одни и те же деревья, она быстро их запомнит, но не сможет распознать новые. Поэтому мы применили динамическое искажение данных (аугментацию). На каждой эпохе обучения программа случайным образом вращала деревья вокруг вертикальной оси, немного меняла их масштаб и добавляла легкий шум к координатам точек. Таким образом, нейросеть видела каждое дерево с сотен разных ракурсов, что заставило ее выучить истинную форму кроны, а не просто заучить координаты. График, иллюстрирующий динамику обучения сети и изменения функции потерь, представлен в *Приложении Б*.

Само обучение состояло из двух фаз, которые повторялись из раза в раз:

- Фаза обучения: пакет искаженных деревьев проходит через сеть. Вычисляется ошибка, и алгоритм рассчитывает, как нужно изменить параметры нейросети. Обновлением параметров управляет специальный оптимизатор, который дополнительно штрафует модель за слишком большие веса, делая ее работу более надежной.
- Фаза проверки: после обновления параметров модель переводится в режим оценки. На этом этапе отключается случайное отключение нейронов, и сеть делает предсказания на чистых деревьях из проверочной выборки. Алгоритм подсчитывает долю правильных ответов.

Чтобы добиться максимальной точности, мы использовали автоматический планировщик скорости обучения. Если программа замечала, что точность на проверочной выборке перестает расти на протяжении нескольких эпох, она плавно уменьшала шаг обучения. Это позволило алгоритму точнее настроить веса сети на финальных этапах.

В коде была реализована стратегия сохранения лучшего результата. Сеть не просто останавливалась после заданного количества циклов, а постоянно сравнивала текущую точность с историческим максимумом. Файл с моделью перезаписывался на жестком диске только в том случае, если сеть ставила новый рекорд распознавания. В итоге мы получили обученную модель, готовую к финальным проверкам на абсолютно новых данных.

3.3. Разработка интерактивного программного интерфейса для визуализации результатов

Трехмерные облака точек — это сложные визуальные объекты, и анализировать их форму только по тексту и цифрам в консоли крайне неудобно. Чтобы наглядно показать, как работают алгоритмы, и иметь возможность глазами проверять деревья на наличие аномалий, прямо в среде разработки был создан графический интерфейс. Он реализован с помощью технологии *Jupyter Widgets*, которая связывает выпадающие меню с кодом на *Python* и позволяет мгновенно отрисовывать деревья на экране.

Разработанный интерфейс решает три основные задачи: визуализацию данных после подготовки, а также интерактивное тестирование модели *Random Forest* и нейросети *PointNet++*.

Основным элементом управления выступает выпадающий список (*Dropdown widget*). Для его наполнения программный скрипт предварительно обходит файловую систему и формирует агрегированный список всех доступных файлов формата «*las*». Для удобства навигации каждый элемент списка содержит префикс, указывающий принадлежность файла к обучающей или отложенной выборке.

При выборе пользователем любого файла из списка срабатывает обработчик событий, который запускает функцию *plot_4_views*. Данная функция выполняет комплексный алгоритм рендеринга:

1) Динамическая предобработка: Выбранный файл пропускается через ту же математическую функцию *process_point_cloud*, которая использовалась при сборке датасета. На этом этапе в реальном времени отсекается рельеф, облако сэмплируется до 2048 точек и нормализуется в единичную сферу. Это позволяет исследователю увидеть данные ровно в том виде, в котором их «видит» математическая модель перед принятием решения.

2) Вычисление оптимального *Bounding Box*: для того, чтобы 3D-модель дерева не искажалась при отрисовке на графике с фиксированными осями, алгоритм вычисляет истинный центр масс облака и максимальный размах координат. Эти вычисления используются для установки жестких и симметричных лимитов для осей (*set_xlim*, *set_ylim*, *set_zlim*) библиотеки *Matplotlib*, обеспечивая изометрию отображения.

3) Отрисовка мультивидовых проекций: для формирования полного представления об архитектуре кроны создается фигура с четырьмя независимыми 3D-осями. С помощью функции изменения угла обзора камеры *view_init(elev, azim)* облако точек последовательно отрисовывается с четырех ракурсов: спереди (*Front*), сбоку (*Side*), сверху (*Top*) и в изометрической проекции (*Isometric*).

4) Цветовое кодирование: Точки на графике окрашиваются градиентом в зависимости от их высоты. Нижняя часть ствола отображается темно-синим цветом, середина кроны – зелено-желтым, а верхушка – красным. Такая раскраска помогает лучше воспринимать объемную структуру дерева на плоском экране монитора.

По аналогии с модулем визуализации, были разработаны отдельные виджеты для тестирования моделей *Random Forest* и *PointNet++*. Их архитектура включает тот же элемент выбора файла, однако фоновая обработка кардинально отличается. Облако точек проходит динамическую предобработку, после чего функция

extract_features_from_cloud мгновенно вычисляет вектор из 17 геометрических признаков. Вектор подается на вход обученной модели через метод *predict_proba*. Полученный результат (вероятностное распределение принадлежности дерева к каждому из классов) форматируется и выводится на экран в виде ранжированного списка.

Виджет для *PointNet++* работает по схожему принципу, но вместо извлечения признаков он преобразует матрицу 2048 точек в вычислительный тензор *PyTorch*, помещает его в батч и пропускает через иерархические сверточные слои сохраненной нейронной сети в режиме без вычисления градиентов (*torch.no_grad()*). Сырые числовые ответы сети нормализуются функцией *Softmax* для перевода в понятные проценты уверенности.

Наличие нескольких независимых меню в одном рабочем окне сильно упрощает анализ. Можно загрузить одно и то же «сложное» дерево во все виджеты одновременно. Сравнивая то, как выглядит дерево глазами человека, с тем, что ответил алгоритм Случайного леса и что предсказала нейросеть *PointNet++*, исследователь может наглядно увидеть, в каких случаях классификаторы ошибаются и какая из моделей справляется с задачей лучше.

4. Анализ результатов и сравнение моделей

4.1. Оценка качества классификации по метрикам Accuracy, Precision, Recall и F1-Score

Для объективной оценки обобщающей способности разработанных алгоритмов тестирование производилось исключительно на независимой отложенной выборке. Данный набор состоял из 24 трехмерных моделей деревьев (ровно по 4 экземпляра для каждого из 6 классов: *Alder*, *Aspen*, *Birch*, *Fir*, *Pine*, *Spruce*), которые были физически изолированы на этапе предобработки и не участвовали в процессах извлечения признаков или настройки весов нейронной сети.

При оценке моделей нельзя опираться только на общий процент правильных ответов. Чтобы понять, где именно ошибается алгоритм, в машинном обучении применяется матрица ошибок (*Confusion Matrix*). На ее основе вычисляются базовые метрики качества, которые используют подсчет верных и ложных срабатываний (TP – истинно положительные, TN – истинно отрицательные, FP – ложно положительные, FN – ложно отрицательные):

1) *Accuracy* (Общая точность) – отражает общую долю корректно классифицированных объектов относительно всего объема тестовой выборки:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2) *Precision* (Точность) – показывает, насколько можно доверять модели, когда она относит дерево к конкретному классу. Это доля реальных деревьев данного вида среди всех деревьев, которым алгоритм присвоил эту породу:

$$Precision = \frac{TP}{TP + FP}$$

3) *Recall* (Полнота) – демонстрирует способность алгоритма обнаруживать заданный класс в принципе. Показывает, какую долю объектов данного класса из всех действительно существующих в выборке модель смогла найти:

$$Recall = \frac{TP}{TP + FN}$$

4) *F1-Score* (F-мера) – среднее гармоническое между точностью (*Precision*) и полнотой (*Recall*). Это самый надежный показатель качества модели, так как он снижается, если алгоритм показывает дисбаланс (например, находит все березы, но при этом часто путает с ними другие деревья):

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Чтобы оценить работу алгоритмов в целом, мы рассчитывали метрики для каждой породы дерева отдельно, а затем выводили их среднее арифметическое. Полные таблицы с отчетами тестирования вынесены в Приложение А.

Для получения максимально честных и объективных результатов мы усложнили финальную проверку. Каждое из 24 скрытых деревьев случайным образом поворачивалось вокруг своей оси 100 раз. В итоге алгоритмам пришлось сделать 2400 предсказаний. Это имитирует ситуацию, когда беспилотник подлетает к дереву с разных сторон, и полностью исключает элемент случайности или удачного ракурса.

Результаты тестирования алгоритма «Случайный лес» продемонстрировали хороший базовый уровень. Итоговая точность составила 88.67%.

Детальный разбор показывает, с какими задачами придуманные нами геометрические признаки справляются хорошо, а с какими – нет. Алгоритм отлично распознает ольху, показывая практически идеальные результаты. Однако серьезные проблемы возникли с березой. Показатель полноты для нее составил всего 0.50. Это значит, что алгоритм нашел лишь половину реальных берёз, спутав остальные с осинами и соснами.

Такие ошибки ожидаемы для классических методов. Общие статистические формулы описывают дерево лишь в общих чертах и не могут уловить уникальный рисунок ветвей. У березы и осины похожие размеры и форма кроны, поэтому их числовые показатели пересекаются, заставляя алгоритм сомневаться. Итоговый показатель качества (*F-мера*) для Случайного леса остановился на отметке 0.88.

Нейронная сеть *PointNet++* справилась с задачей заметно лучше. Ее итоговая точность достигла 91.58%.

Анализ отчета из *Приложения А* показывает, что нейросеть выучила гораздо более сложные пространственные закономерности. В отличие от Случайного леса, модель уверенно отличает хвойные породы друг от друга, достигая высоких показателей точности на елях, пихтах и соснах (от 0.94 до 0.98). Сеть научилась анализировать не просто высоту, а локальные сгустки хвои и текстуру ствола.

Главная сложность для нейросети по-прежнему кроется в паре лиственных пород – осине и березе. Несмотря на то, что показатель полноты для березы вырос

до 0.90, точность распознавания осины немного просела. Эти деревья визуально похожи даже для алгоритмов глубокого обучения, и при вращении облака точек их переплетающиеся ветви порой образуют идентичные узоры.

Тем не менее, общий показатель качества (F-мера) для нейросети составил 0.92, уверенно обойдя базовую модель. Это доказывает, что способность алгоритмов самостоятельно искать нужные признаки прямо в 3D-координатах работает эффективнее, чем ручное составление математических формул.

4.2. Визуальный анализ результатов, интерпретация признаков и сравнительная оценка моделей

Чтобы наглядно увидеть, как именно мыслят алгоритмы и где они допускают промахи, результаты проверки были переведены в графики. Для анализа мы построили матрицы ошибок, график важности признаков и итоговую диаграмму.

Матрица ошибок – это лучший инструмент для поиска слабых мест классификатора. Она показывает, с какими именно породами модель путает конкретное дерево. Результаты масштабной проверки (на 2400 предсказаниях) представлены на рисунке ниже.

Если посмотреть на матрицу ошибок *Random Forest* (Рис. 1.), видно, что основная масса правильных ответов лежит на главной диагонали, но модель работает нестабильно. Взглянув на строку березы (*Birch*), можно увидеть причину падения метрик: из 400 тестовых показов модель угадала березу лишь 200 раз. В 100 случаях она назвала ее осиной (*Aspen*), и еще в 100 случаях – сосной (*Pine*). Классическая математика просто не смогла найти уникальные черты этих деревьев при смене ракурса.

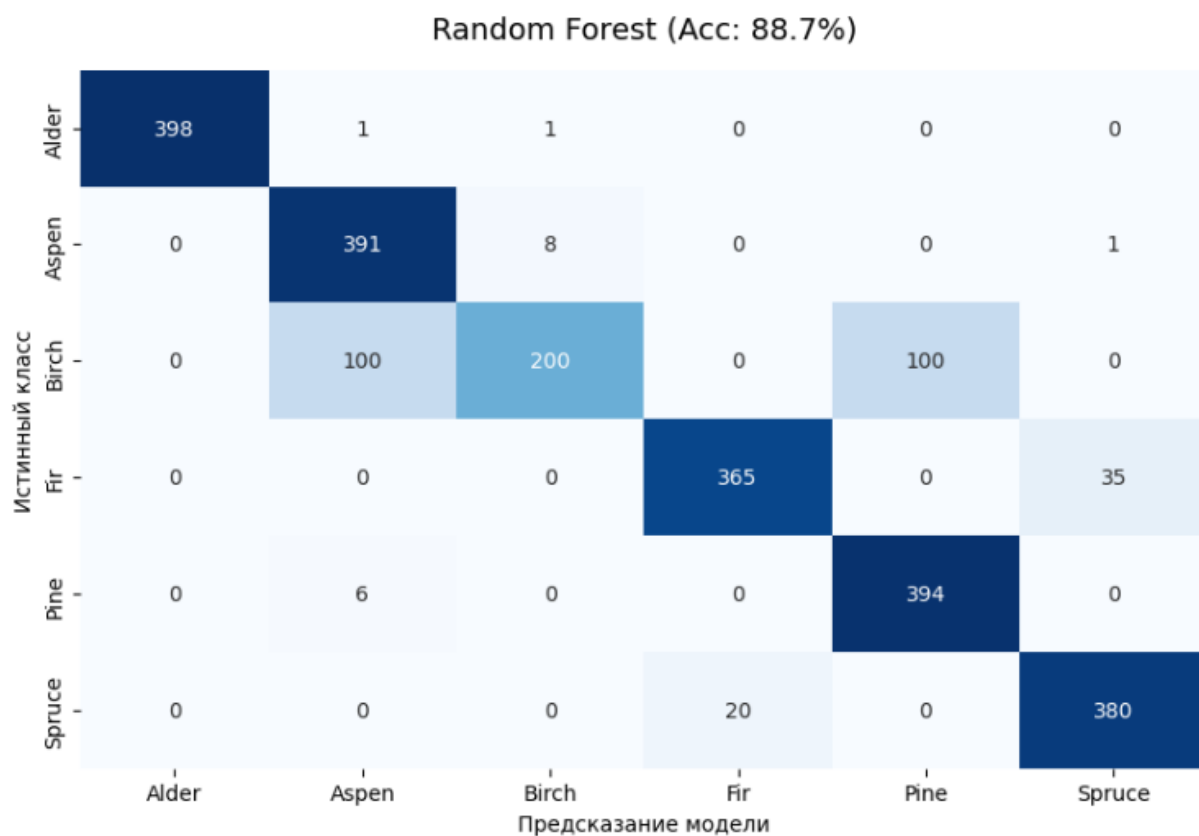


Рис. 1. Матрица ошибок Random Forest

Матрица ошибок *PointNet++* выглядит гораздо увереннее. Диагональ плотная и темная, а путаница между породами сведена к минимуму. Алгоритм, основанный на анализе локальных участков, практически безошибочно разделит хвойные деревья. Ошибки сконцентрированы лишь в биологически родственных классах: сеть иногда принимает березу за ольху или сосну, но общее количество таких промахов в разы меньше, чем у первой модели.

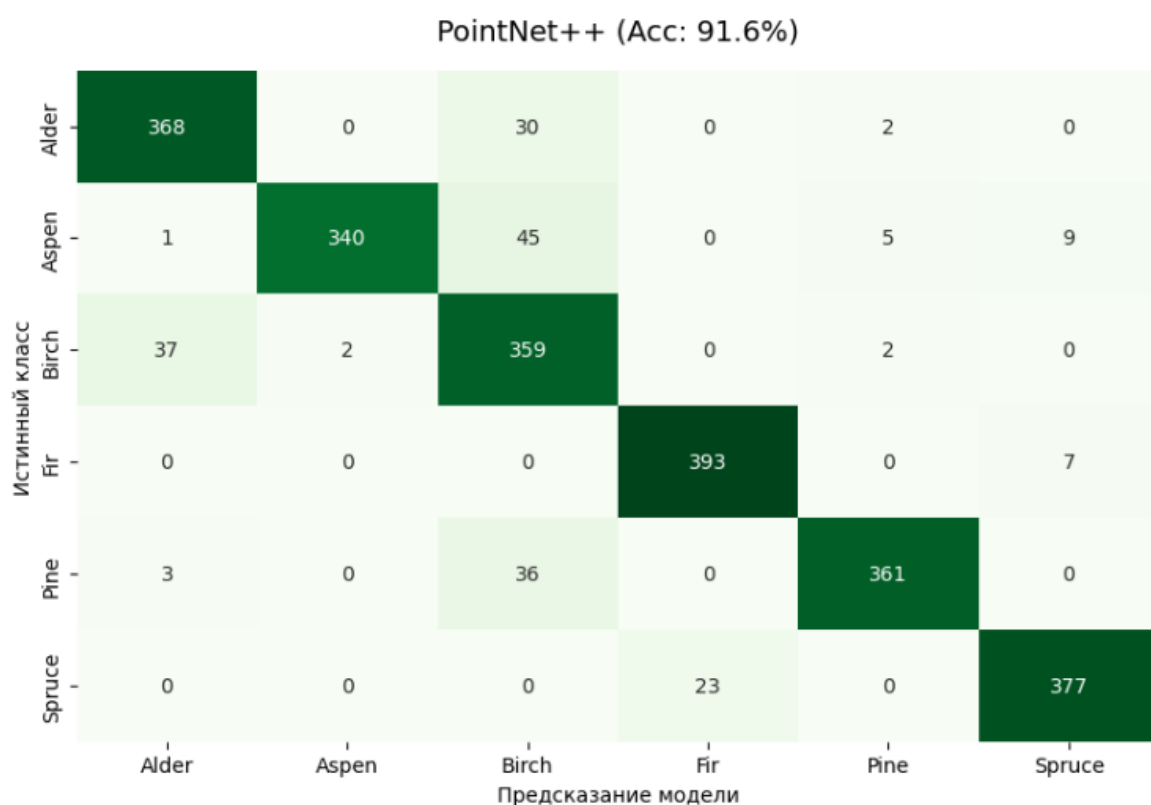


Рис. 2. Матрица ошибок PointNet++

Одно из немногих преимуществ классического Случайного леса перед нейросетями – его прозрачность. Глубокие сети работают скрыто, и мы не можем точно узнать, на какие именно изгибы веток смотрела программа, принимая решение. Случайный лес, напротив, позволяет точно измерить пользу каждого параметра, заданного человеком.

Горизонтальная диаграмма важности признаков вынесена в *Приложение В*. Анализ этого графика позволяет сделать несколько интересных выводов о геометрии леса:

- Важность середины дерева: самым ценным признаком оказалась плотность точек в средней трети ствола. Именно этот параметр позволяет легко отсеять хвойные деревья (у которых середина густая) от лиственных (где средняя часть ствола часто голая до начала основной кроны).

- Важность профиля: в топе полезных признаков закрепились уровни распределения высот. Для компьютера высота, на которой сосредоточена основная масса веток, оказалась гораздо важнее, чем просто общая ширина кроны.
- Бесполезность простого разброса: признаки, описывающие общую дисперсию, оказались самыми бесполезными. Они сильно зависят от качества сканирования и мало говорят об истинной форме дерева.

Итоговое количественное сравнение двух подходов представлено на столбчатой диаграмме в *Приложении В*.

Диаграмма наглядно иллюстрирует превосходство глубокого обучения по всем ключевым параметрам. Общая точность возросла на три процента, а способность находить все деревья нужного класса без пропусков (полнота) увеличилась с 88.7% до 91.6%. Главный показатель баланса – F-мера – вырос на 4 процента. Эти цифры, полученные в условиях жесткого стресс-теста со случайным вращением объектов, доказывают высокую надежность нейросетевого подхода для инвентаризации лесных массивов.

Заключение

В ходе выполнения курсовой работы была успешно решена актуальная научно-практическая задача автоматической классификации деревьев Дальнего Востока на основе сырых трехмерных данных лазерного сканирования (*LiDAR*). Работа охватила полный цикл создания аналитической системы: от математической очистки пространственных координат до обучения и глубокого сравнительного анализа двух концептуально разных алгоритмов.

На первом этапе был разработан надежный алгоритм подготовки данных. Применение порогового фильтра позволило аккуратно отсечь рельеф земли и шумовой подлесок, оставив для анализа исключительно надземную часть дерева. Последующее приведение всех облаков к единому размеру (2048 точек) и нормализация масштаба обеспечили корректную передачу данных в алгоритмы. Важным шагом стало выделение полностью независимой отложенной выборки.

Изоляция этих файлов от процесса обучения гарантировала, что финальная проверка моделей будет честной и объективной.

Для проведения вычислительного эксперимента были протестированы и сопоставлены два подхода:

- Метод извлечения геометрических признаков (*Random Forest*). Трехмерное облако точек преобразовывалось в набор из 17 понятных человеку параметров (габариты, распределение высот, дисперсия). Данный подход продемонстрировал хорошую базовую точность на уровне 88.67%. Его главным преимуществом стала прозрачность: анализ показал, что густота ветвей в средней части ствола и профиль кроны являются ключевыми факторами для определения породы. Однако этот метод оценивает дерево лишь по общим статистическим габаритам, из-за чего совершает критические ошибки при попытке различить схожие деревья (например, березу и осину) при изменении угла обзора.

- Глубокое обучение (*PointNet++*). В отличие от первой модели, нейросеть обучалась напрямую на трехмерных координатах, без использования ручных формул. Архитектура сети была усложнена для лучшего запоминания деталей, а в процесс обучения была внедрена динамическая аугментация (вращение и искажение данных). Это позволило модели самостоятельно изучить уникальную микроструктуру ветвей и ствола. Результаты жесткого стресс-тестирования показали существенный прирост качества: итоговая точность составила 91.58%. Нейросеть доказала свою устойчивость к смене ракурсов и практически полностью исключила путаницу среди хвойных пород.

Проведенное исследование позволяет сделать однозначный вывод: традиционные методы, основанные на ручном подборе геометрических параметров, уступают современным нейросетевым подходам. Способность глубокого обучения напрямую анализировать 3D-данные и выучивать сложную локальную структуру веток делает его наиболее эффективным, точным и перспективным инструментом для автоматизированной инвентаризации обширных лесных ресурсов региона.

Полученные метрики Random Forest для каждой породы дерева:

Random Forest	Precision	Recall	F1-Score	Количество тестовых файлов
Ольха	1.00	0.99	1.00	400
Осина	0.79	0.98	0.87	400
Береза	0.96	0.50	0.66	400
Пихта	0.95	0.91	0.93	400
Сосна	0.80	0.98	0.88	400
Ель	0.91	0.95	0.93	400
Accuracy			0.89	2400
Macro-Average	0.90	0.89	0.88	2400

Полученные метрики PointNet++ для каждой породы дерева:

PointNet++	Precision	Recall	F1-Score	Количество тестовых файлов
Ольха	0.90	0.92	0.91	400
Осина	0.99	0.85	0.92	400
Береза	0.76	0.90	0.83	400
Пихта	0.94	0.98	0.96	400
Сосна	0.98	0.90	0.94	400
Ель	0.96	0.94	0.95	400
Accuracy			0.92	2400
Macro-Average	0.92	0.92	0.92	2400

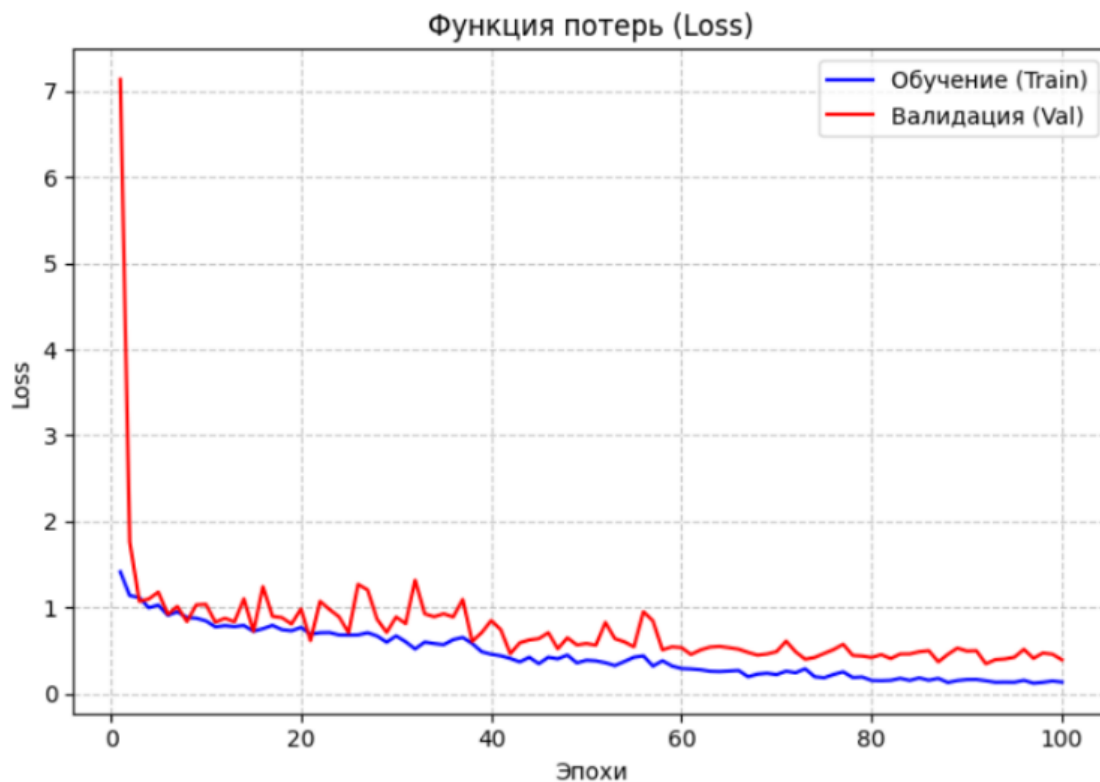


График изменения функции потерь при обучении PointNet++



График изменения точности на тренировочной и валидационной выборках при обучении PointNet++

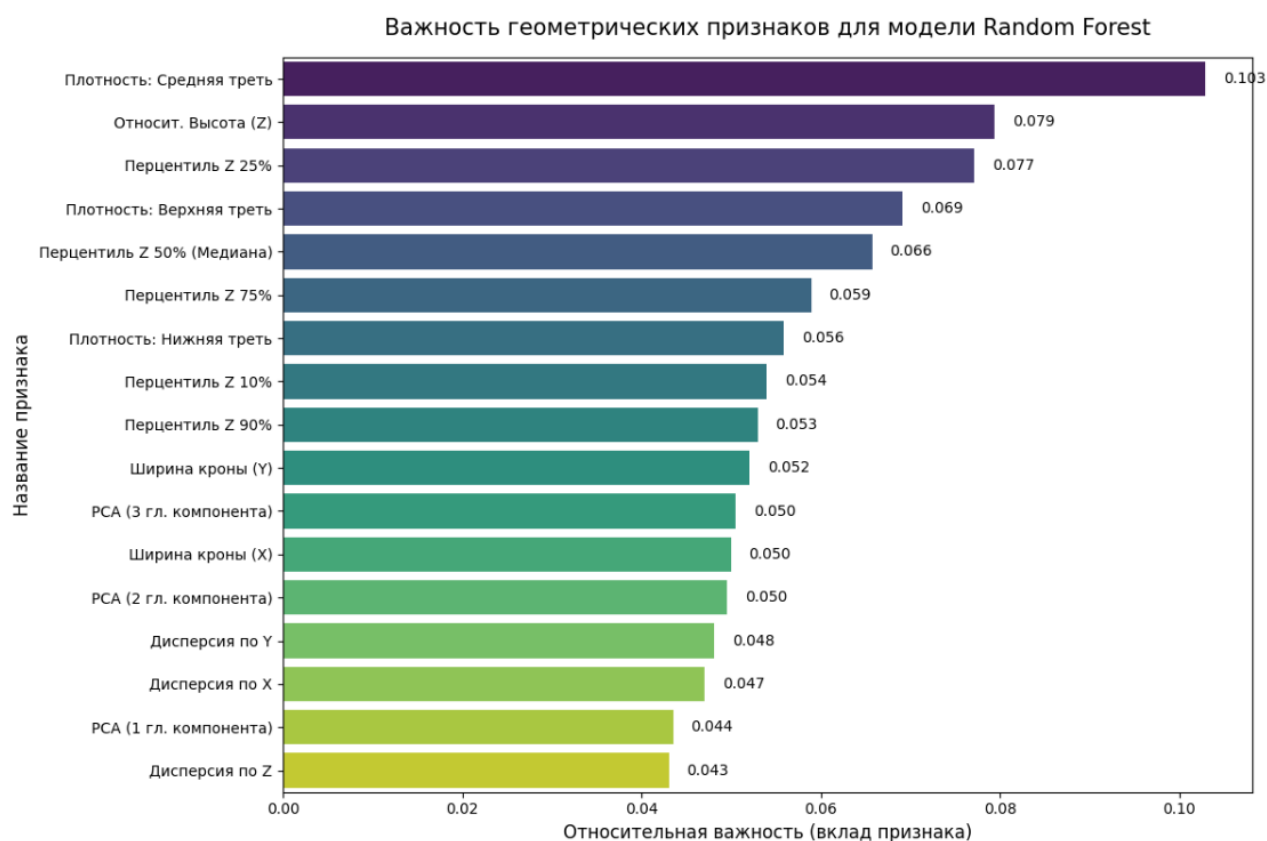


Диаграмма важности признаков в Random Forest

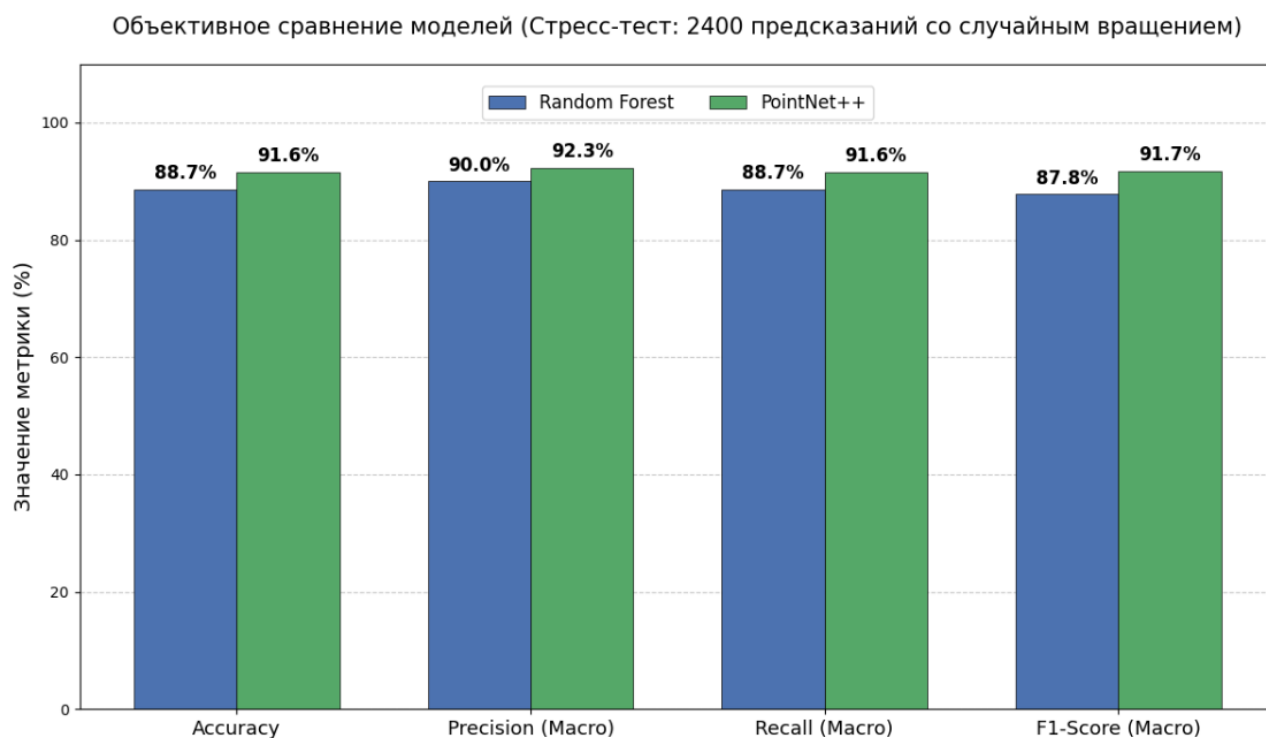


Диаграмма сравнения двух моделей машинного обучения